

CSE-303

COMPILER

Language Processor & Structure of a Compiler

Maruf Ahmed Tamal
Assistant Professor
Department of Computer Science and Engineering
Military Institute of Science and Technology (MIST)

First, share your thoughts

WHAT IS A PROGRAMMING LANGUAGE



PROGRAMMING LANGUAGE

A programming language is a set of rules, symbols, and instructions used to communicate with computers

PROGRAMMING LANGUAGE EXIST

Category	Approximate number
All historical / experimental languages	~9,000
Languages currently in some use	200–1,300+
Common professional languages	**30–50**
Very popular languages	**10–20**

TOP PROGRAMMING LANGUAGES

Jun 2026	Jun 2025	Change	Programming Language	Ratings	Change
1	1		Python	18.96%	-6.91%
2	3	↑	C	10.77%	+1.30%
3	2	↓	C++	8.03%	-2.65%
4	4		Java	7.90%	-0.94%
5	5		C#	4.85%	+0.17%
6	6		JavaScript	3.04%	-0.17%
7	8	↑	Visual Basic	2.80%	+0.59%
8	12	⇈	SQL	1.77%	+0.23%
9	14	⇈	R	1.69%	+0.30%
10	9	↓	Delphi/Object Pascal	1.54%	-0.60%

Source: TIOBE Programming Community index

LOW VS HIGH LEVEL PROGRAMMING LANGUAGES

LOW LEVEL LANGUAGE

A low-level language allows programmers to write instructions that **directly control the computer's hardware**, such as the CPU, memory, and registers.

HIGH LEVEL LANGUAGE

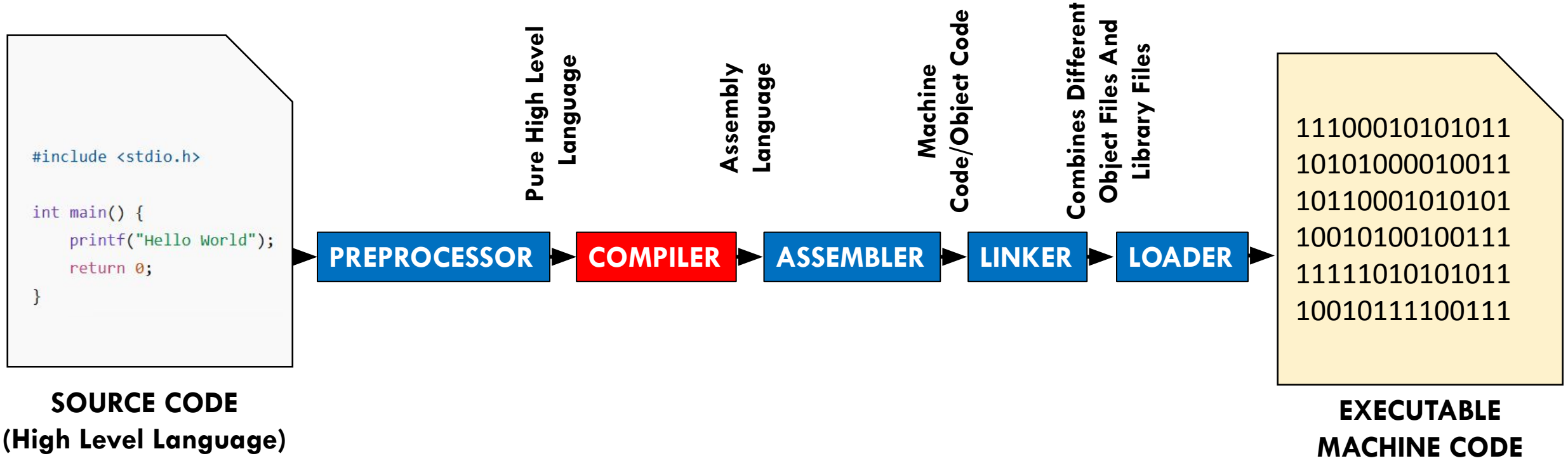
A high-level programming language allows programmers to write instructions using **English-like words and logical structures** instead of directly writing binary or machine code.

LOW VS HIGH LEVEL PROGRAMMING LANGUAGES

Basis of Comparison	Low-Level Language	High-Level Language
Ease of understanding	Difficult to read and write	Easy to read and write
Human friendliness	Less human-friendly	More human-friendly
Machine friendliness	More machine-friendly	Less directly machine-friendly
Speed	Usually faster	Usually slightly slower than low-level languages
Hardware control	Gives direct control over hardware	Hides hardware details
Portability	Less portable; depends on machine/processor	More portable; can run on different systems
Translation needed	Needs assembler or direct machine execution	Needs compiler or interpreter (language processing system)
Error finding	More difficult	Easier
Development time	Takes more time	Takes less time
Examples	Machine language, Assembly language	Python, Java, C++, JavaScript, PHP
Main use	Operating systems, embedded systems, device drivers	Web apps, mobile apps, AI, business software

LANGUAGE PROCESSING SYSTEM

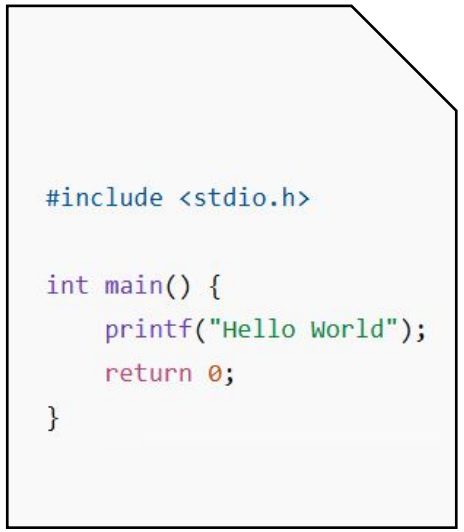
A Language Processing System is **a collection of programs** that translate, process, and prepare source code for execution by the computer.



LANGUAGE PROCESSING SYSTEM

(SOURCE PROGRAM)

The source program is the code written by the programmer



```c
#include <stdio.h>

int main() {
    printf("Hello World");
    return 0;
}
```

SOURCE CODE
(High Level Language)

Element	Example
Header files	#include <stdio.h>
Main function	int main()
Statements	printf("Hello World");
Variables	int a, b;
Logic	loops, conditions, functions

LANGUAGE PROCESSING SYSTEM

(PREPROCESSOR)

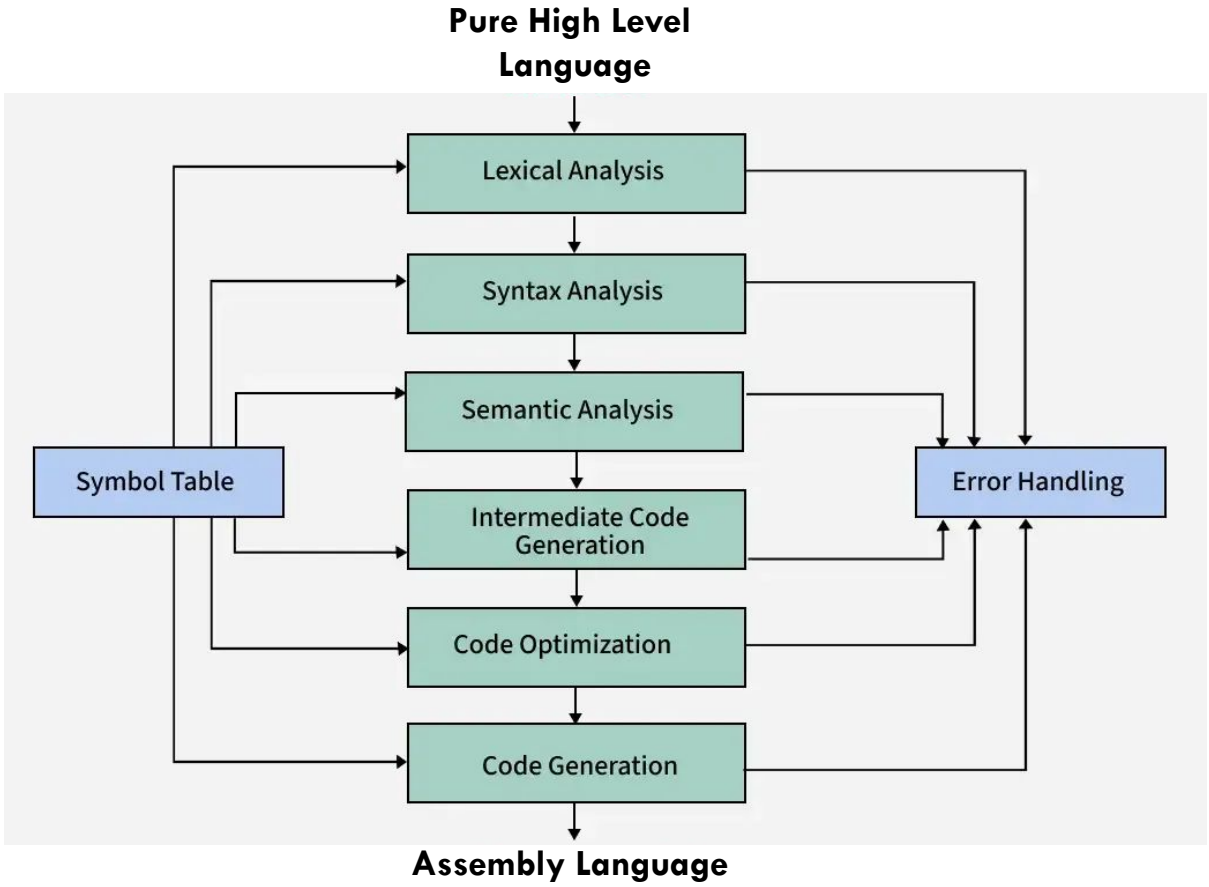
The preprocessor works before the compiler. It handles lines that usually begin with #. It prepares the program for the compiler.

Functionality	Explanation	Example
Header file inclusion	Adds necessary library information into the program	#include <stdio.h>
Macro expansion	Replaces macro names with their values	#define PI 3.1416
Conditional compilation	Compiles selected parts of code based on conditions	#ifdef, #ifndef
Removing comments	Removes comments from the source code	// comment
Preparing expanded code	Produces an expanded source file for the compiler	Source code after preprocessing

LANGUAGE PROCESSING SYSTEM

(COMPILER)

The compiler translates high-level code into lower-level code, usually assembly code or object code. It is one of the most important parts of the language processing system.



Functionality	Explanation
Checks syntax errors	Finds grammar mistakes in the program
Checks semantic errors	Finds meaning-related mistakes
Translates code	Converts high-level code into assembly/object code
Optimizes code	Makes code faster and more efficient
Generates target code	Produces assembly or object code

LANGUAGE PROCESSING SYSTEM

(COMPILER)

The compiler has several internal phases:

Phase	Main Function	What Happens in This Phase	Example
1. Lexical Analysis	Breaks the program into tokens	The source code is scanned character by character. Meaningful units called tokens are created. It removes spaces, comments, and unnecessary symbols.	int sum; ⟶ int, sum, ;
2. Syntax Analysis	Checks grammar of the program	It checks whether the tokens follow the grammar rules of the programming language. It creates a parse tree or syntax tree.	Checks whether sum = a + b; is grammatically correct

LANGUAGE PROCESSING SYSTEM

(COMPILER)

Phase	Main Function	What Happens in This Phase	Example
3. Semantic Analysis	Checks meaning of the program	It checks data types, variable declarations, function calls, and logical meaning. It ensures that the program makes sense.	int a; a = "Hello"; → semantic error
4. Intermediate Code Generation	Converts source code into intermediate form	The compiler generates an intermediate code that is neither fully high-level nor fully machine-level. This makes optimization easier.	sum = a + b → t1 = a + b; sum = t1;
5. Code Optimization	Improves the code	The compiler improves the intermediate code so that the program runs faster and uses less memory. It removes unnecessary operations.	x = 5 * 2; may become x = 10;

LANGUAGE PROCESSING SYSTEM

(COMPILER)

Phase	Main Function	What Happens in This Phase	Example
6. Code Generation	Produces target code	The optimized intermediate code is converted into assembly code or machine-level code.	MOV R1, a / ADD R1, b
7. Error Handling	Reports errors	The compiler detects and reports different errors found during compilation. It helps programmers fix mistakes.	Missing semicolon, undeclared variable
8. Symbol Table Management	Stores important information	The compiler keeps a table of variable names, function names, data types, memory locations, and scope information.	int age; → name: age, type: int

LANGUAGE PROCESSING SYSTEM

(ASSEMBLER)

The assembler converts assembly language into machine code/object code.

Functionality	Explanation
Converts assembly to machine code	Changes symbolic instructions into binary machine instructions
Creates object file	Produces object code such as .obj or .o file
Handles symbols	Converts labels and symbolic names into memory addresses
Detects assembly errors	Finds errors in assembly instructions

The assembler produces object code, but the program is not always ready to run yet. It may still need library functions.

LANGUAGE PROCESSING SYSTEM

(LINKER)

The actual code of printf() comes from the standard library. The linker connects our program with that library code.

Functionality	Explanation
Combines object files	Joins multiple object files into one program
Links library functions	Connects functions like printf(), scanf(), sqrt()
Resolves external references	Finds missing function definitions or variable references
Creates executable file	Produces final executable program such as .exe

LANGUAGE PROCESSING SYSTEM

(LOADER)

The loader loads the executable program into main memory/RAM.

Functionality	Explanation
Loads executable into memory	Places the program in RAM
Allocates memory	Gives memory space for variables, stack, and heap
Adjusts addresses	Converts logical addresses into actual memory addresses
Starts execution	Transfers control to the program’s starting point

LANGUAGE PROCESSING SYSTEM

(EXECUTION)

In the execution step, the CPU starts running the program.

Functionality	Explanation
CPU executes instructions	Runs machine code line by line
Input is accepted	Takes data from keyboard, file, or device
Processing is done	Performs calculation or logic
Output is produced	Shows result on screen or stores it
Program terminates	Ends after completing execution


Thank You!



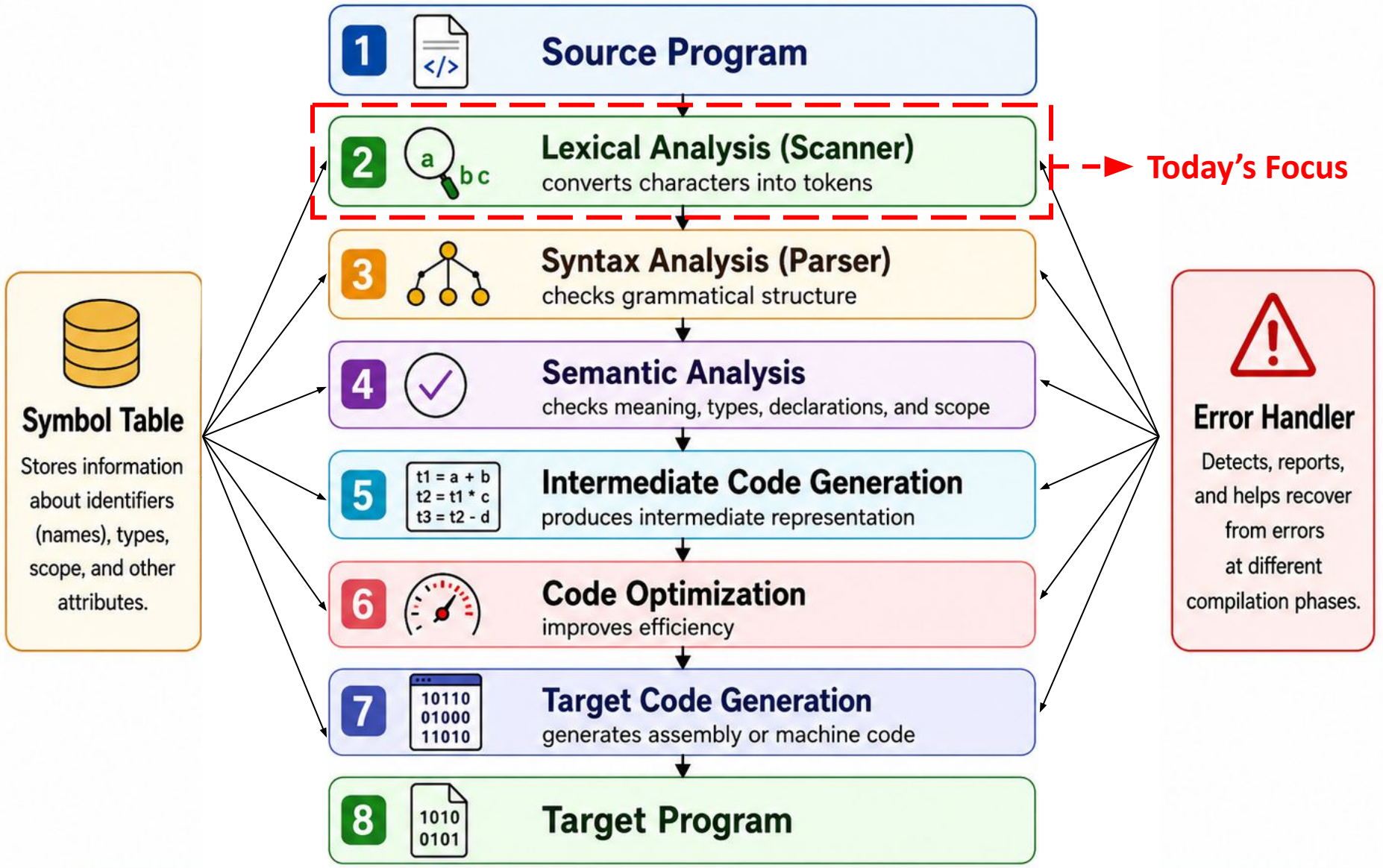
CSE-303

COMPILER

Lexical Analysis: The First Phase of the Complier

Maruf Ahmed Tamal
Assistant Professor
Department of Computer Science and Engineering
Military Institute of Science and Technology (MIST)

PHASES OF COMPILER

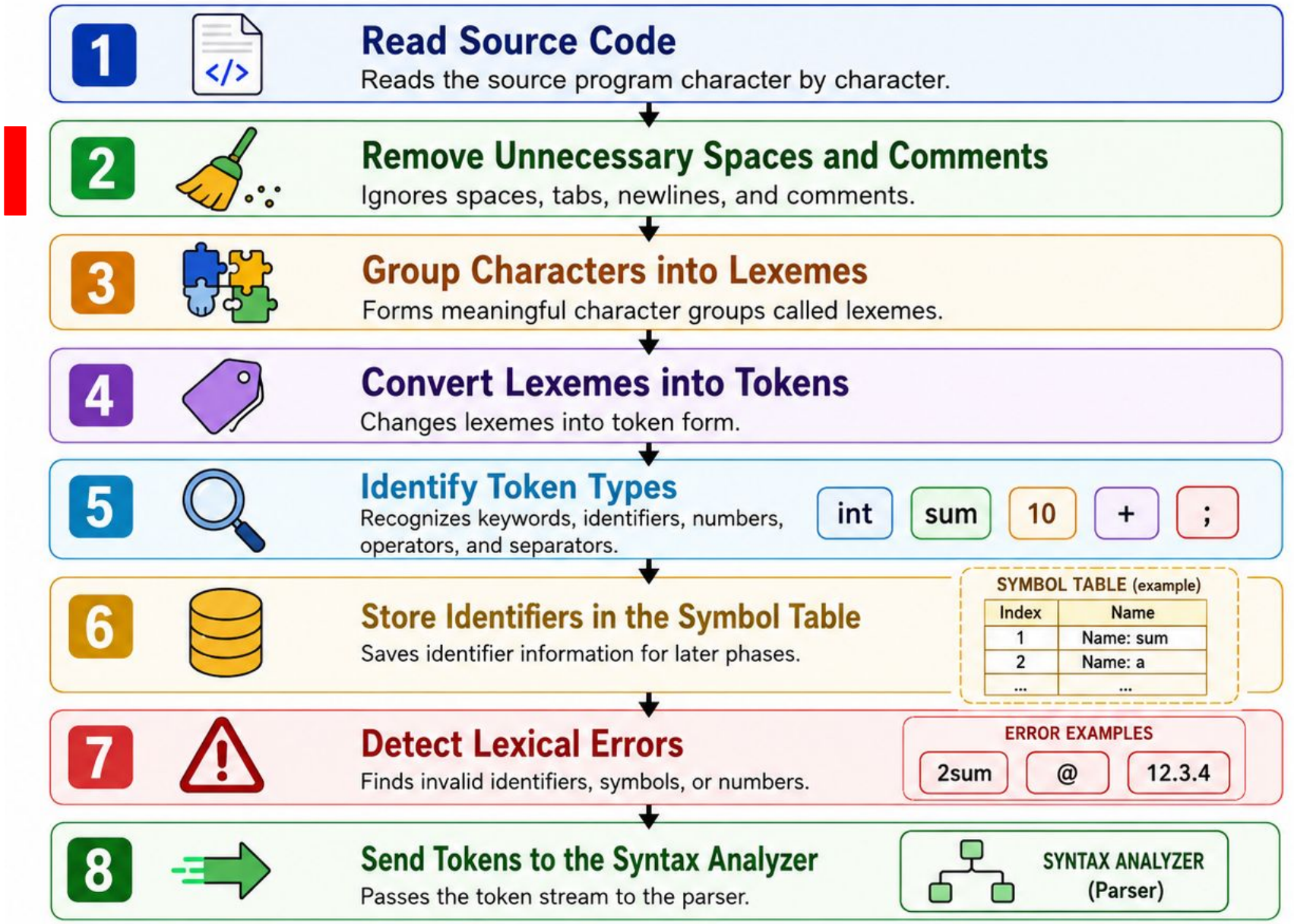


LEXICAL ANALYSIS

Lexical Analysis is the **first phase** of a compiler that reads the pure high level language (source code) as **a sequence of characters** and converts it into **meaningful units called tokens**

The **lexical analyzer**, also called a **scanner**, takes the raw source code and breaks it into **smaller parts** such as keywords, identifiers, operators, constants, and punctuation symbols.

OBJECTIVES OF LEXICAL ANALYSIS



HOW LEXICAL ANALYSIS WORKS

Before understanding how Lexical Analysis works, we need to understand some important terms clearly. These terms are:

Term	Meaning	Simple Example	Explanation
Character	The smallest unit of source code	s, u, m, =, 1, 0, ;	The compiler first reads the program character by character.
Lexeme	The actual word or symbol written in the program	int, sum, =, 10, ;	Lexemes are formed by grouping characters.
Token	The category or class of a lexeme	KEYWORD, IDENTIFIER, NUMBER	Token tells what type of lexeme it is.
Pattern	The rule used to identify a token	letter(letter/digit)*	Pattern defines how a valid lexeme should look.
Symbol Table	A table that stores information about identifiers	sum → int	It stores variable/function names, type, scope, value, etc.

HOW LEXICAL ANALYSIS WORKS

STEP 1: INPUT SOURCE PROGRAM (PURE HIGH LEVEL LANGUAGE)

A. The lexical analyzer receives the source code from the previous stage.

```
int sum = a + 10;
```

B. It reads the program as a stream of characters:

```
i n t   s u m   =   a   +   1 0 ;
```

STEP 2: REMOVE UNNECESSARY CHARACTERS

A. The lexical analyzer ignores characters that are not needed for token generation. It removes/skips: extra spaces, tabs, newlines, comments.
Note: spaces are not always useless. They help separate words.

HOW LEXICAL ANALYSIS WORKS

STEP 3: GROUP CHARACTERS INTO LEXEMES

A lexeme is the actual sequence of characters found in the program.

Lexemes:

```
int
sum
=
a
+
10
;
```

So, in this step the lexical analyzer groups characters into meaningful parts.

HOW LEXICAL ANALYSIS WORKS

STEP 4: MATCH LEXEMES WITH TOKEN PATTERNS

Each lexeme is checked against predefined rules or patterns.

- Each token pattern is usually written using Regular Expression.
- DFA helps the lexical analyzer decide whether a sequence of characters matches a valid token pattern.

Concept	Meaning	Role in Lexical Analysis
RE	Pattern/rule	Defines token patterns
NFA	Machine with possible multiple paths	Intermediate form created from regular expression
DFA	Machine with one fixed path for each input	Used for fast token recognition
Lexical Analyzer	Scanner	Uses DFA to identify lexemes and generate tokens

HOW LEXICAL ANALYSIS WORKS

STEP 4: MATCH LEXEMES WITH TOKEN PATTERNS

Pattern	Example
Keyword	int, float, if, while
Identifier	sum, count, total
Number	10, 25, 3.14
Operator	+, -, =, *
Separator	;, ,, (,)

HOW LEXICAL ANALYSIS WORKS

STEP 5: GENERATE TOKENS

After identifying the lexemes, the lexical analyzer generates tokens (based on the pattern).

Lexeme	Token
int	<KEYWORD, int>
sum	<IDENTIFIER, sum>
=	<ASSIGN_OP, =>
a	<IDENTIFIER, a>
+	<ADD_OP, +>
10	<NUMBER, 10>
;	<SEMICOLON, ;>

Question: How Lexical Analyzer Knows int is Keyword and sum is Identifier?

Answer: The compiler has a predefined list of keywords.

HOW LEXICAL ANALYSIS WORKS

STEP 6: ENTER IDENTIFIERS INTO SYMBOL TABLE

The lexical analyzer also helps create or update the symbol table.

A symbol table is a data structure used by the compiler to store information about identifiers such as variable names, function names, data types, scope, memory location, and other attributes.

At lexical analysis stage, the analyzer mainly stores the name of the identifier. Later phases such as semantic analysis update more details.

Name	Type	Value	Scope
sum	int	10	local/global

HOW LEXICAL ANALYSIS WORKS

STEP 7: DETECT LEXICAL ERRORS

The lexical analyzer also detects invalid tokens.

```
int 2sum = 10;
```

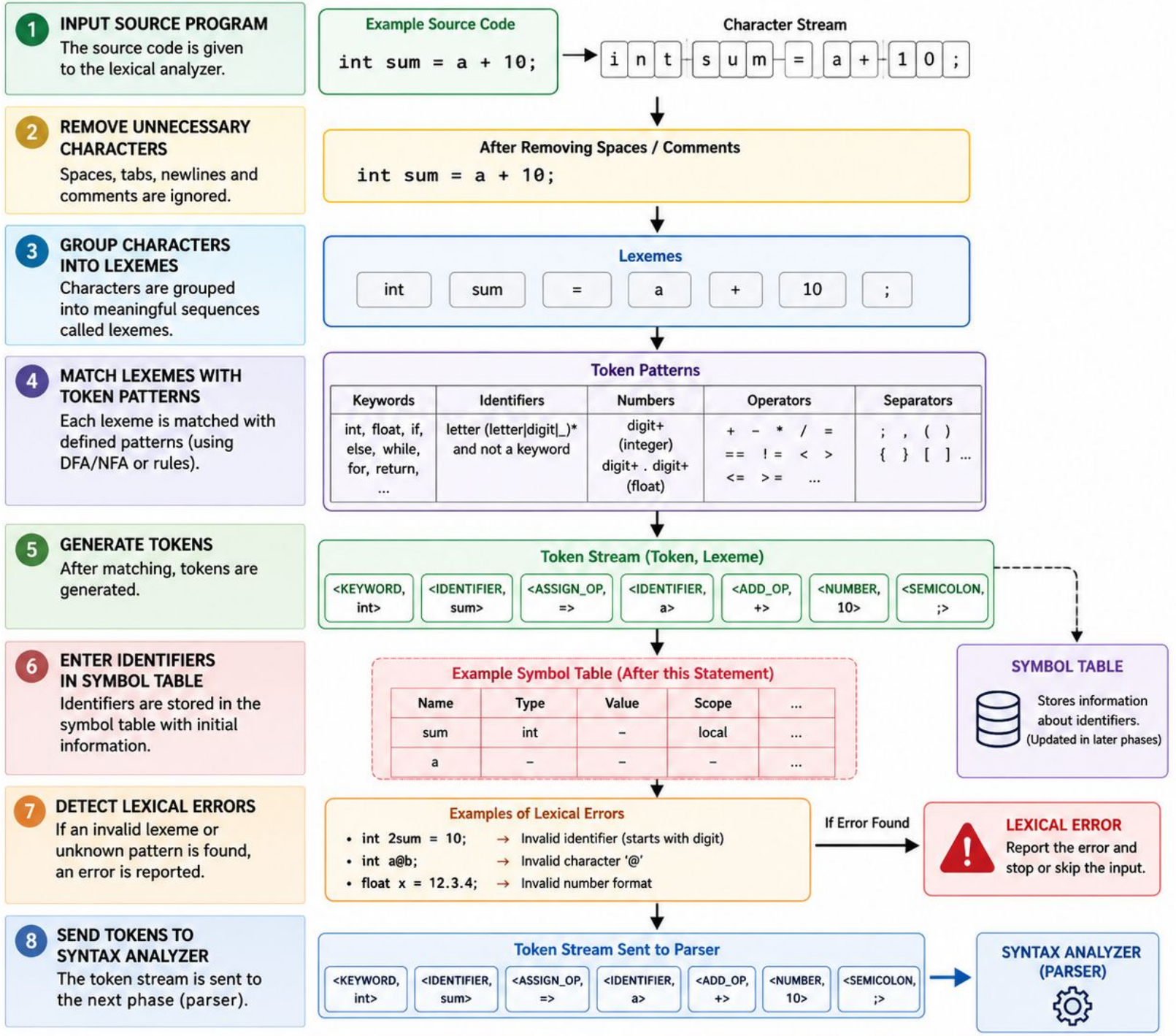
```
int a@b;
```

STEP 8: SEND TOKENS TO SYNTAX ANALYZER

After generating tokens, the lexical analyzer sends them to the next phase:

<KEYWORD, int>
<IDENTIFIER, sum>
<ASSIGN_OP, =>
<IDENTIFIER, a>
<ADD_OP, +>
<NUMBER, 10>
<SEMICOLON, ;>

HOW LEXICAL ANALYSIS WORKS AT A GLANCE



RE, NFA, & DFA IN COMPILER DESIGN

Regular Expression (RE): A Regular Expression is a formal notation used to describe the pattern of a token.
In compiler design, regular expressions define the structure of tokens such as: Keywords, Identifiers, Numbers, Operators, Separators.

For example, the pattern of an identifier can be written as:

$$(letter \mid _)(letter \mid digit \mid _)^*$$

This means:

A. The first character must be a letter or underscore.
B. The remaining characters may be letters, digits, or underscores.

RE, NFA, & DFA IN COMPILER DESIGN

Nondeterministic Finite Automaton (NFA): An NFA is a finite-state machine constructed from a regular expression.

In an NFA:

- A state may have more than one transition for the same input.
- A state may have an ε-transition.
- An ε-transition allows the machine to change state without reading any input character.
- Several possible paths may exist for recognizing the same string.

For example, an NFA for an identifier may contain different paths for: Letters, Digits, Underscores. The NFA accepts the input when at least one possible path reaches an accepting state.

In compiler design, the NFA is mainly an intermediate representation between the regular expression and the DFA.

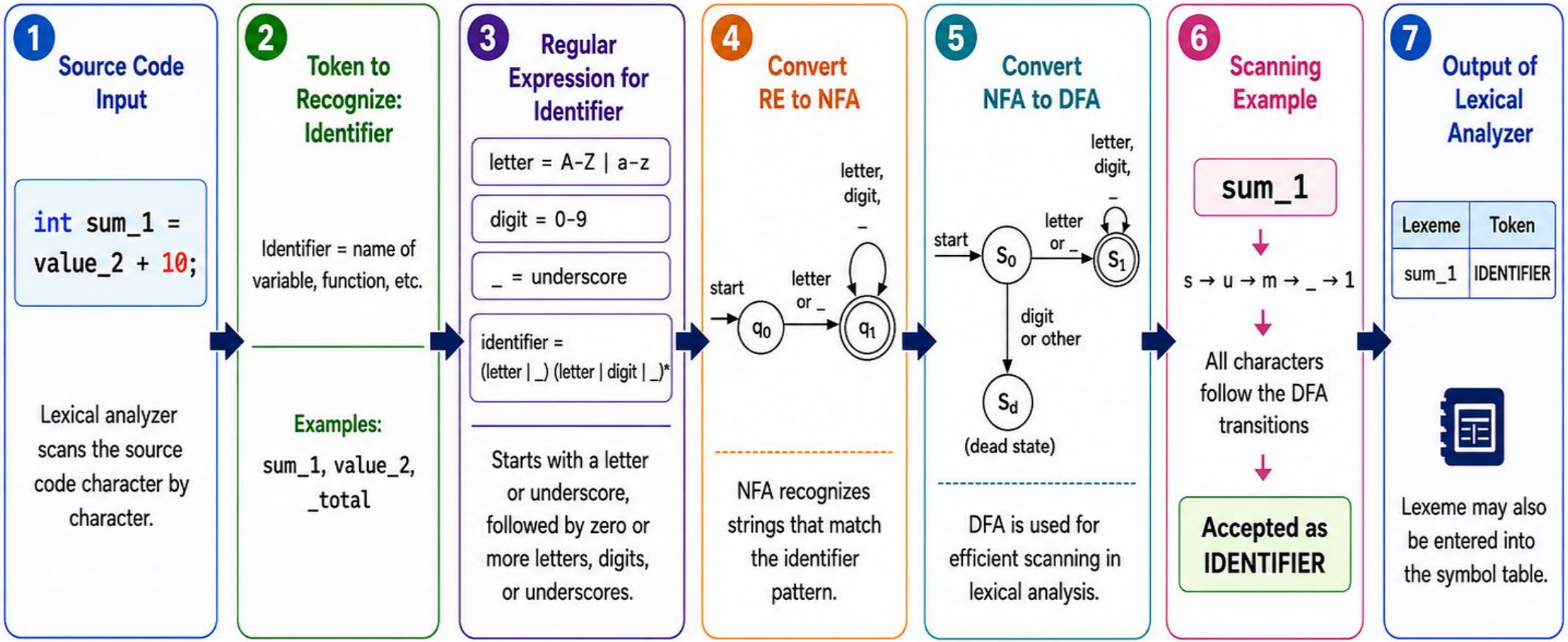
RE, NFA, & DFA IN COMPILER DESIGN

Deterministic Finite Automaton (DFA): A DFA is a finite-state machine in which only one transition is possible for each input character from a particular state.

In a DFA:

- There are no ε-transitions.
- Each state has only one transition for a particular input.
- The machine follows one definite path.
- Token recognition is fast and efficient.

HOW A LEXER RECOGNIZES A IDENTIFIER USING RE, NFA, & DFA



HOW A LEXER RECOGNIZES A KEYWORD USING RE, NFA, & DFA

1 Source Code Input

```
int sum = 10;
```

Lexical analyzer scans the source code character by character.

2 Token to Recognize: Keyword

Keyword = reserved word with predefined meaning in a programming language.

Examples:

`int, float, if, else, while`

3 Regular Expression for Keyword

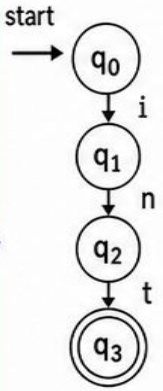
`keywords = int | float | if | else | while`

Example keyword = `int`

A keyword is matched exactly as a reserved word.

4 Convert RE to NFA

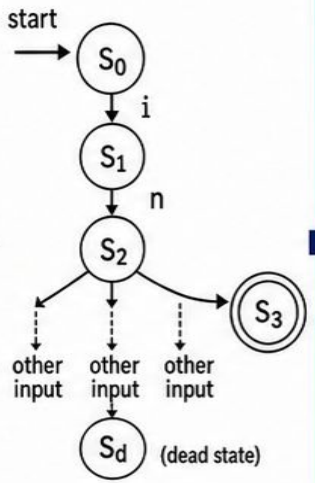
Example keyword: int



NFA recognizes the exact keyword pattern.

5 Convert NFA to DFA

Example keyword: int



DFA is used for efficient scanning in lexical analysis.

6 Scanning Example

`int`

i → n → t

All characters follow the DFA transitions

Accepted as KEYWORD

7 Output of Lexical Analyzer

Lexeme	Token
int	KEYWORD

Lexeme may also be checked in the reserved-word table.

RE Regular Expression defines the pattern → **NFA** NFA represents the pattern → **DFA** DFA performs fast recognition → Lexical Analyzer outputs the token.


Thank You!



CSE-303

COMPILER

Lexical Analysis: The First Phase of the Complier

Maruf Ahmed Tamal
Assistant Professor
Department of Computer Science and Engineering
Military Institute of Science and Technology (MIST)

INPUT BUFFER IN LEXICAL ANALYSIS

Input buffering is a technique used in lexical analysis to read source code efficiently.

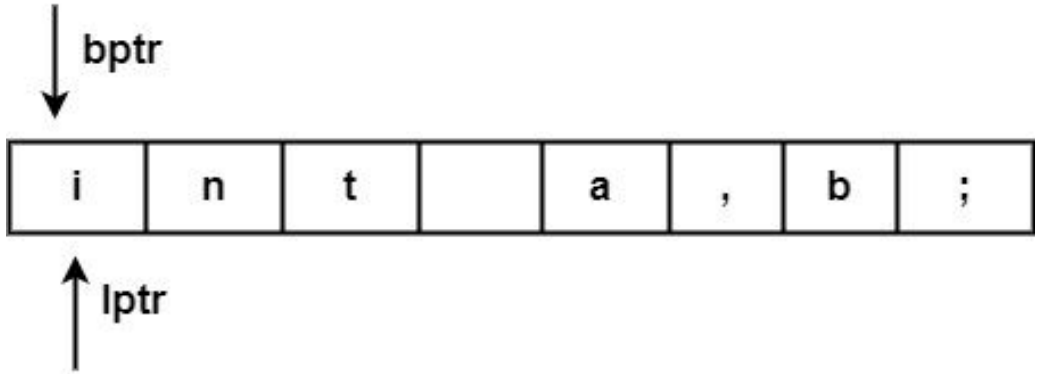
A. Lexical Analysis has to access secondary memory each time to identify tokens. It is time-consuming and costly.
B. Instead of reading one character directly from the file every time, the lexical analyzer loads a block of characters into a temporary memory area called a buffer.
C. The lexer then scans characters from that buffer to recognize tokens such as keywords, identifiers, numbers, and operators.

INPUT BUFFER IN LEXICAL ANALYSIS

Lexical Analysis scans input string from left to right one character (from buffer) at a time to identify tokens. It uses two pointers to scan tokens –

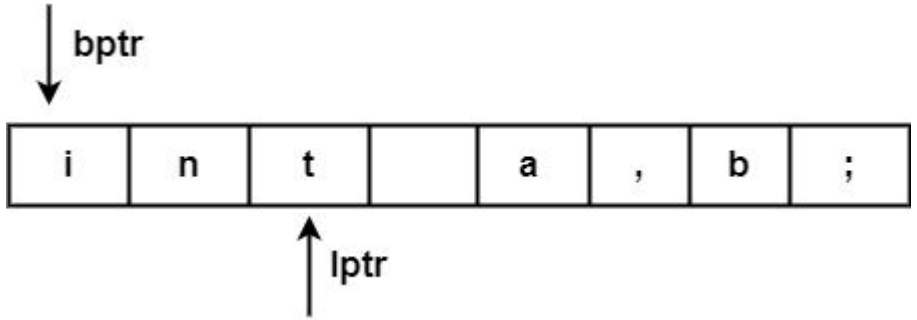
Begin Pointer (bptr) – It points to the beginning of the string to be read.
Look Ahead Pointer (lptr) – It moves ahead to search for the end of the token.
Example – For statement **int a, b;**

A. Both pointers start at the beginning of the string, which is stored in the buffer.

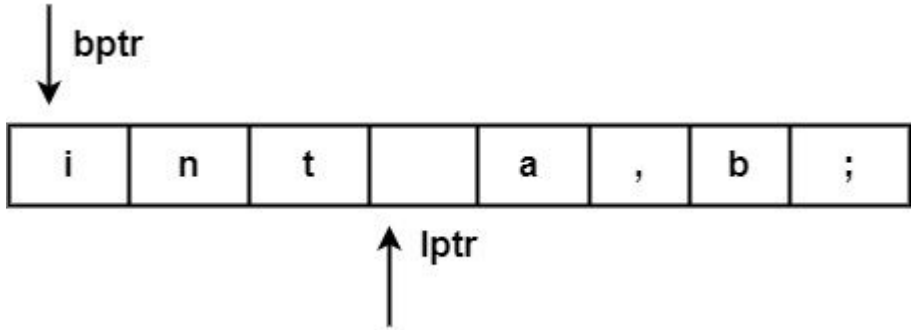


INPUT BUFFER IN LEXICAL ANALYSIS

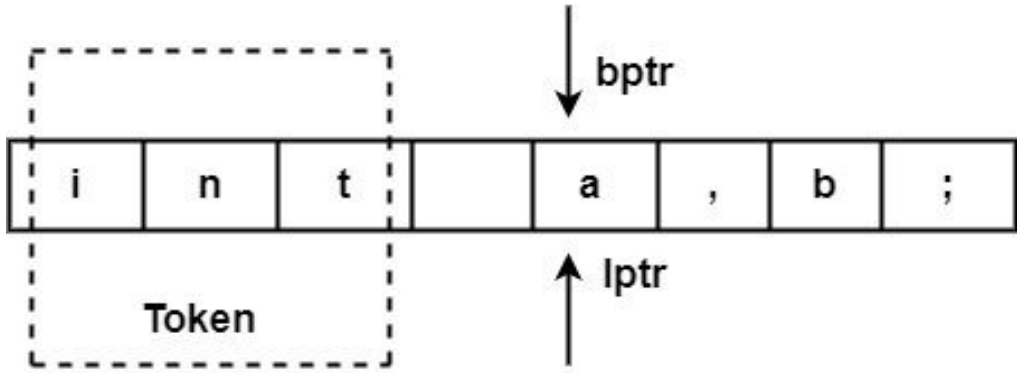
B. Look Ahead Pointer scans buffer until the token is found.



C. The character ("blank space") beyond the token ("int") have to be examined before the token ("int") will be determined.

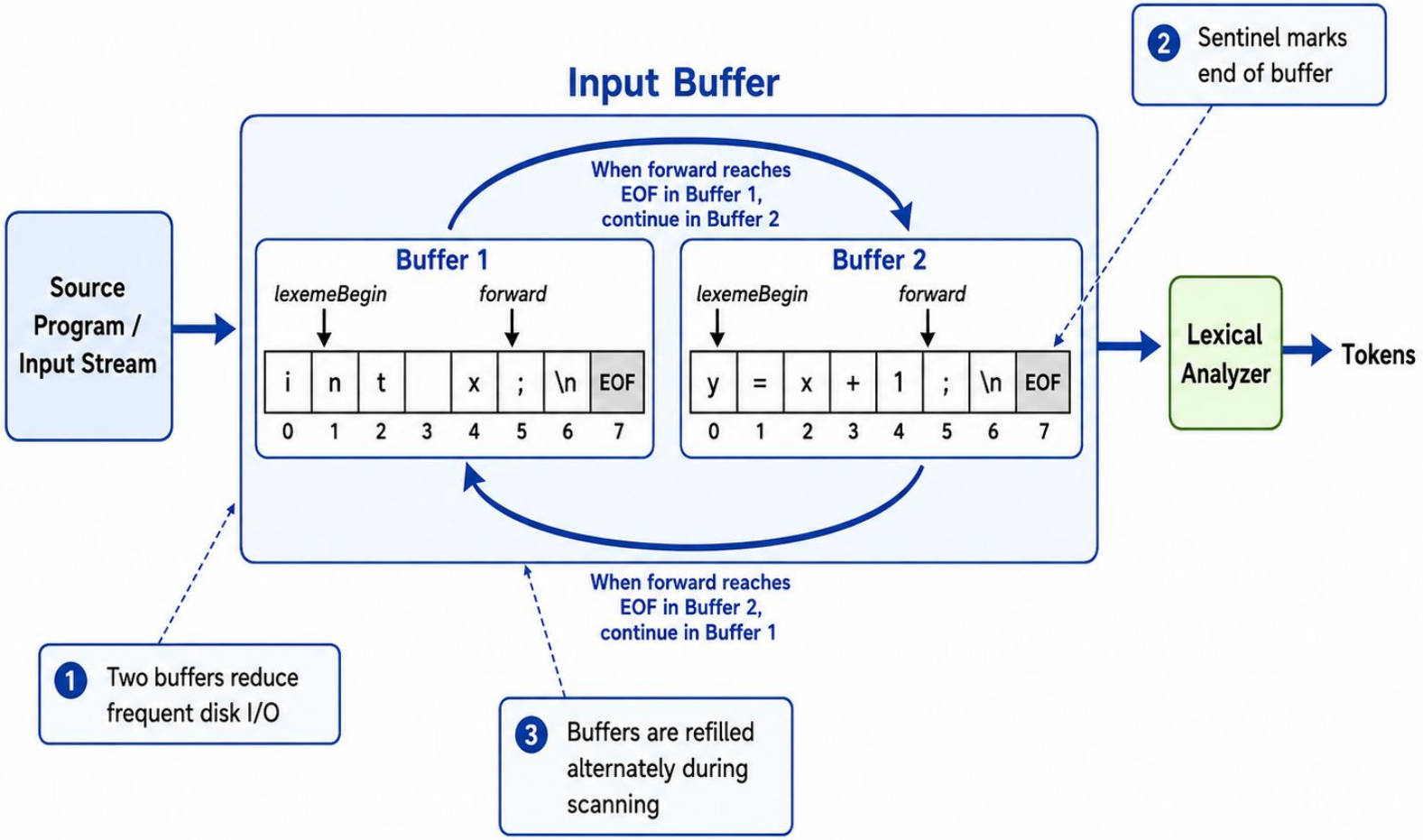


D. After processing token ("int") both pointers will set to the next token ('a'), & this process will be repeated for the whole program.



INPUT BUFFER IN LEXICAL ANALYSIS

A buffer can be divided into two halves. If the look Ahead pointer moves towards halfway in First Half, the second half is filled with new characters to be read. If the look Ahead pointer moves towards the right end of the buffer of the second half, the first half will be filled with new characters, and it goes on.



Sentinels – Sentinels are used to making a check, each time when the forward pointer is converted, a check is completed to provide that one half of the buffer has not converted off. If it is completed, then the other half should be reloaded.

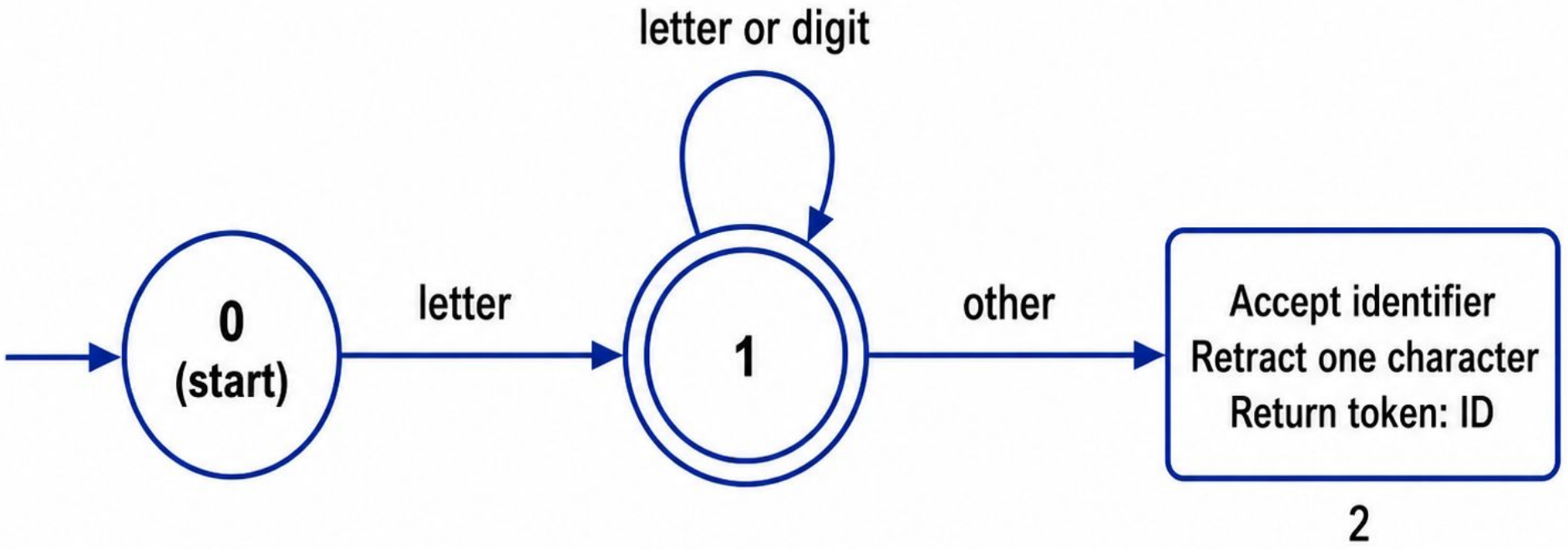
TRANSITION DIAGRAM

A transition diagram in compiler design is a graphical representation of how the lexical analyzer recognizes tokens by moving from one state to another based on input characters.

A transition diagram typically consists of:

- **States** are represented by circles.
- **Transitions** are represented by arrows.
- **Input symbols or conditions** are written on the arrows.
- The **starting state** has an incoming arrow.
- **An accepting or final state** is shown using a double circle.

TRANSITION DIAGRAM



Identifier pattern:

letter (letter | digit)*

Examples:

x, sum1, count23


Thank You!